

AI CODING EVOLUTION

From Autocomplete to Autonomous Agents

The Evolution of AI in Development



The journey from simple syntax completion to intelligent agents that can autonomously write, test, and debug code

Traditional Tools

Static analysis, rule-based suggestions, limited context awareness

AI-Powered Tools

Learning from millions of examples, understanding context, generating complete solutions autonomously

Six Major Eras

01

2010-2015

Basic Autocomplete

02

2016-2019

ML-Powered Suggestions

03

2021-2022

GitHub Copilot Era

04

2023

Conversational Coding

05

2024

Targeted Code Editing

06

2025+

Autonomous Agents

Era 1: Basic Autocomplete

2010-2015

0
1



Syntax Completion

Auto-complete keywords and identifiers based on language rules



Local Context

Suggestions limited to current file scope and imported modules



Rule-Based

Static analysis without machine learning or pattern recognition



IntelliSense

Microsoft's implementation became the industry standard

The ML Revolution

02

ML-Powered Suggestions (2016-2019)



TabNine & Kite introduced ML models trained on millions of code examples, predicting entire lines based on patterns

03

GitHub Copilot Era (2021-2022)



OpenAI Codex revolutionized coding with AI pair programming - generating functions from natural language



Function
Generation



Natural
Language



Context-Aware



Real-time

Interactive Development

04

Conversational Coding (2023)



ChatGPT, Claude, and Cursor enable developers to discuss architecture, debug, and refactor through dialogue

05

Targeted Code Editing (2024)



AI edits specific code sections inline while preserving context, style, and multi-file relationships

Interactive Dialogue

Code Explanation

Inline Edits

Multi-file Context

Era 6: Autonomous Agents

2025+

0
6



AI agents autonomously complete development tasks with minimal human intervention



Autonomous Execution

Complete tasks independently from planning to deployment



Testing & Debugging

Run tests, identify failures, and fix automatically



Tool Integration

Interact with terminals, APIs, databases, and more



Multi-step Workflows

Execute complex sequences across multiple tools



Self-Correction

Learn from errors and adjust approach iteratively

Key Insights

EVOLUTION DRIVERS

- ✓ Larger training datasets
- ✓ Better language models
- ✓ Improved context windows
- ✓ Multi-modal capabilities
- ✓ Tool integration

FUTURE CHALLENGES

- ✗ Code quality consistency
- ✗ Security vulnerabilities
- ✗ Dependency management
- ✗ Testing completeness
- ✗ Human oversight needs



QUICK STARTED WITH GITHUB COPILOT

Build a complete task manager app while learning AI coding

What You'll Build & Learn



A hands-on tutorial building a complete task management web application

The Application

- ✓ Add, delete, and complete tasks
- ✓ Filter system (all/completed/pending)
- ✓ Modern responsive design
- ✓ Dark mode toggle

AI Skills You'll Master

- ✓ Inline suggestions as you type
- ✓ Autonomous feature building
- ✓ Precise code editing
- ✓ AI personalization & smart actions

Prerequisites:

✓ VS Code installed

✓ GitHub Copilot access (free tier available)

Step 1: Inline Suggestions

AI-powered suggestions appear as you type



Ghost text suggestions help you write code faster with fewer errors

Try It:

1. Create `index.html` file
2. Start typing: `<!DOCTYPE html>`
3. Press `Tab` to accept the ghost text suggestion
4. Continue building your HTML structure

Navigation:

Alt +]

Next suggestion

Alt + [

Previous
suggestion

Tab

Accept suggestion

0
1

Step 2: Build with Agents

Autonomous feature development across multiple files



Give natural language prompts - agents plan and implement complex features

How to Use Agents:

- 1 Open Chat view (Ctrl+Alt+I or click chat icon)
- 2 Select 'Agent' mode from the agent picker
- 3 Enter your prompt and let the agent work
- 4 Review changes and click 'Keep' to accept

Example Prompt:

"Create a complete task manager web application with the ability to add, delete, and mark tasks as completed. Include modern CSS styling and make it responsive. Use semantic HTML and ensure it's accessible. Separate markup, styles, and scripts into their own files."

Step 3: Inline Chat

Make precise adjustments to specific code sections

0
3

Agents

Large features across multiple files

Inline Chat

Targeted improvements within a file

How to Use Inline Chat:

- 1 Select the code block you want to improve
- 2 Press Ctrl+I to open inline chat
- 3 Enter your prompt (e.g., 'Add input validation')
- 4 Review changes and select 'Keep' or 'Discard'

Perfect For:

Adding error handling

Refactoring functions

Fixing specific bugs

Step 4: Personalize AI

Customize chat to match your coding style

0
4

Custom Instructions

Create [.github/copilot-instructions.md](#)

Define your coding preferences:

- Code style (ES6+, semantic HTML)
- Naming conventions (camelCase, PascalCase)
- Code quality (error handling, comments)

```
# Project general coding guidelines
```

```
## Code Style
```

- Use semantic HTML5 elements (header, main, section, article, etc.)
- Prefer modern JavaScript (ES6+) features like const/let, arrow functions, and template literals

```
## Naming Conventions
```

- Use PascalCase for component names, interfaces, and type aliases
- Use camelCase for variables, functions, and methods
- Prefix private class members with underscore (_)
- Use ALL_CAPS for constants

```
## Code Quality
```

- Use meaningful variable and function names that clearly describe their purpose
- Include helpful comments for complex logic
- Add error handling for user inputs and API calls

Step 4: Personalize AI

0
4

Customize chat to match your coding style



Custom Agents

Create specialized AI personas for specific tasks

Example: 'Code Reviewer' agent

- Reviews code without making changes
- Provides analysis and suggestions
- Follows your project standards

```
---
description: 'Review code for quality and adherence
            to best practices.'

tools: ['usages', 'vscodeAPI', 'problems', 'fetch',
        'githubRepo', 'search']

---
# Code Reviewer agent

You are an experienced senior developer conducting
a thorough code review.
Your role is to review the code for quality, b
est practices, and adherence
to [project standards](../copilot-instructions.md)
without making direct code changes.
When reviewing code, structure your feedback with clear
headings and specific examples from the code being reviewed.

## Analysis Focus
- Analyze code quality, structure, and best practices
- Identify potential bugs, security issues,
  or performance problems
- Evaluate accessibility and user experience considerations

## Important Guidelines
- Ask clarifying questions about design decisions
  when appropriate
- Focus on explaining what should be changed and why
- DO NOT write or suggest specific code changes directly
```

Step 5: Smart Actions

Pre-built AI integrated into your workflow



AI appears contextually where you need it - no chat required

Example: AI-Generated Commit Messages

- 1 Open Source Control view (Ctrl+Shift+G)
- 2 Stage your changes
- 3 Click the sparkle icon ✨
- 4 AI analyzes changes and generates descriptive message
- 5 Review and commit or regenerate

Other Smart Actions:

Debugging assistance

Test generation

Code explanation

Your Learning Journey

Progressive skill building from basics to advanced

01

Inline Suggestions

Write HTML structure

02

Agents

Build complete task manager

03

Inline Chat

Add validation & features

04

Personalization

Set coding standards

05

Smart Actions

Generate commit messages



CHAT

Organize conversations and maximize AI productivity



MANAGE CHAT SESSIONS

Organize conversations and maximize AI productivity

What is a Chat Session?



A chat session is your conversation history with AI, including all prompts, responses, and context

Context Window

Each session maintains its own conversation history and context

Follow-Up Questions

Session history enables AI to understand your follow-up prompts

Topic Isolation

Start new sessions to avoid mixing unrelated contexts

💡 Start new sessions when switching topics to get clearer, more focused responses

Three Ways to Create Sessions

Choose the best location for your workflow

01

Chat View

Cmd+N / Ctrl+N

Side panel - keeps Primary Side Bar free for Explorer/Source Control

Best for: Quick questions while browsing code

02

Editor Tab

New Chat Editor

Opens alongside code files - arrange side-by-side with your work

Best for: Multiple sessions or comparing solutions

03

Separate Window

New Chat Window

Floating window - perfect for multi-monitor setups

Best for: Keep chat visible on second monitor

Chat Session History

Return to previous conversations and continue where you left off



All chat sessions are automatically saved - never lose your conversation history

How to Access History:

- 1 Click 'Show Chats...' in the Chat view title bar
- 2 Select any previous session to restore it
- 3 All prompts, responses, and context are restored

Why Use History:

- ✓ Review past interactions and decisions
- ✓ Reference previous AI responses
- ✓ Resume interrupted conversations

Save & Export Sessions

Preserve important conversations for later reference or reuse

0
2

Export as JSON File

Command: 'Chat: Export Chat...'

Saves all prompts and responses to a JSON file

Perfect for: Sharing with team, documentation, archiving

Save as Reusable Prompt

Type: '/savePrompt' in chat input and press Enter

Creates a .prompt.md file with placeholders

Perfect for: Reusing workflows, team templates

Quick Copy Options:

Right-click any message → Copy | Right-click Chat view → Copy All (Markdown)

Context-Isolated Subagents

Delegate tasks to isolated agents within your session

0
3



Subagents operate autonomously with their own context window, then return only final results

Perfect for: Research, analysis, multi-step tasks that need isolation

How to Invoke Subagents:

- 1 Enable the 'runSubagent' tool in the tool picker
- 1 Ask AI to use a subagent in your prompt
- 1 Subagent completes task autonomously
- 1 Receives only the final result in main session

Example Prompts:

"Use a subagent to research best auth methods. Summarize findings."

Agent Sessions (Preview)

Manage background AI tasks across different environments

0
4

Centralized view for all active chat sessions - both local and remote coding agents

Supported Integrations:

✓	Local VS Code sessions	Your interactive chats
✓	GitHub Copilot coding agent	Background code generation
✓	GitHub Copilot CLI	Terminal-based assistance
✓	OpenAI Codex	Pro+ subscription access

Delegate Tasks:

From any local chat session, click 'Delegate to Agent' to offload complex tasks while you continue working

Best Practices

Tips for managing chat sessions effectively

Start New for Different Topics

Avoid context contamination - get clearer responses

Use Editor Tabs for Comparisons

Compare solutions side-by-side with multiple sessions

Separate Windows for Multi-Monitor

Keep chat visible on secondary screen while coding

Background Tasks with Remote Agents

Offload time-consuming tasks to coding agents

Interactive for Real-Time

Use local sessions when you need immediate feedback



MANAGE CONTEXT FOR AI

Get more relevant and accurate AI responses

Why Context Matters



Context is the information AI uses to understand and respond to your questions

More Relevant Responses

AI understands your specific codebase and project context

Accurate Solutions

Responses are tailored to your actual files and code structure

Better Follow-ups

AI remembers conversation history and references

Four Ways to Add Context:

Workspace Indexing • Implicit Context • # Mentions • @ Mentions

Workspace Indexing

Fast and accurate codebase search

0
1

Remote Index

GitHub-hosted index for cloud-based codebases

- Quick search for large repos
- Hosted on GitHub servers
- Best for teams

Local Index

Advanced semantic index stored on your machine

- Fast local search
- Works offline
- Privacy-focused

Basic Index

Simple algorithms for local search

- Fallback option
- Optimized for large codebases
- No setup required

Implicit Context

Automatically included based on your activity



VS Code automatically adds context from your current work - no manual setup needed

Automatically Included:

- 1 Selected text in the active editor
- 2 File name or notebook name of active editor
- 3 Active file (when using Ask or Edit mode)
- 4 Agent-determined context based on your prompt

💡 Implicit context works behind the scenes - you don't need to do anything!

Mentions

Explicitly add files, tools, and web content

0
3

Type # in chat to see available context items

#file

Reference specific files, folders, or symbols

"Fix the bug in #auth.ts"

#codebase

Search entire workspace for relevant code

"How does authentication work? #codebase"

#fetch

Retrieve content from web pages

"What's new in VS Code 1.100 #fetch <url>- https://code.visualstudio.com/updates/v1_100"

#githubRepo

Search code in GitHub repositories

"How does routing work? #githubRepo vercel/next.js"

@ Mentions

Domain-specific experts for specialized questions

0
4



Chat participants are specialized assistants optimized for specific domains

Built-in Chat Participants:

@vscode

VS Code settings and features

"@vscode how to enable word wrap"

@terminal

Terminal commands and shell tasks

"@terminal top 5 largest files"

@workspace

Codebase-specific questions

"@workspace explain this project structure"

Key Difference:

mentions = add context | @ mentions = specialized assistants

Vision & Browser Elements

Advanced context capabilities

Vision Capabilities

Attach images to your chat prompts and ask questions about them



Explain code from screenshots



Implement UI from sketches



Debug visual issues

Browser Elements (Experimental)

Select elements from Simple Browser and add them as context

1. Enable `chat.sendElementsToChat.enabled` setting
2. Open Simple Browser (or Live Preview extension)
3. Click Start to begin selecting elements
4. Click elements to add as context

Best Practices

Tips for effective context management



DO



Be Specific with Files

Add exact files with # instead of relying on #codebase

```
"Fix bug in #auth.ts"
```



Use #codebase for Discovery

When you don't know which files are relevant

```
"How does auth work? #codebase"
```



Combine Multiple Types

Mix #files, #fetch, and @participants

```
"Update to .NET 9 #fetch <url> #Program.cs"
```



DON'T



Overload with Context

Too many files dilutes AI focus

```
"Fix #file1 #file2 #file3 #file4 #file5..."
```



Mix Unrelated Topics

Start new sessions for different subjects

```
[Asking about CSS after debugging Python]
```



Forget to Add Context

Vague questions get vague answers

```
"How does this work?" (What is 'this'?)
```



Pro Tip: Start with specific files, then expand to #codebase if needed

Quick Reference

Context cheat sheet

#

Files, folders, symbols, tools

Explicit context control

@

Chat participants

Domain-specific questions

#codebase

Workspace search

Find relevant code automatically

#fetch

Web content

Reference documentation or examples



Drag & drop

Quick file attachment



Images

Visual context (screenshots, sketches)



REVIEW **AI-GENERATED CODE** **EDITS**

Accept, reject, or refine AI changes with confidence

How AI Code Edits Work



AI generates code changes across multiple files - applied immediately but require your review

1

AI Makes Changes

Edits are applied and saved to disk immediately

2

VS Code Tracks Edits

Pending changes are marked with squared-dot icon

3

You Review & Decide

Keep, undo, or modify individual or all changes

4

Persist Across Sessions

Pending edit status saved when you close VS Code



Changes are saved immediately - but you stay in control!

Pending Changes

Track and identify files with AI edits

0
1



Chat View

Shows complete list of edited files



Explorer View

Squared-dot icon next to modified files



Editor Tabs

Squared-dot indicator on tab name



Editor Content

Inline diff showing old vs new code

Persistence:

Close and reopen VS Code - pending edit status is remembered and restored

Review Changes

Navigate and control AI edits

0
2

Editor Overlay Controls:

↑ Up / ↓ Down

Navigate between suggested edits

Keep Button

Accept edits for the file

Undo Button

Reject edits for the file

Inline Change Controls:

Hover over individual changes to accept/reject specific edits within a file

Chat View Controls:

Accept or reject ALL changes across ALL files at once

Source Control Integration

Seamless integration with Git workflow

0
3



AI edits work seamlessly with your existing Git workflow



Stage Changes

→ Pending edits automatically accepted



Discard Changes

→ Pending edits automatically discarded

Auto-Accept Edits

Automate acceptance with countdown timer

0
4



Configure VS Code to automatically accept edits after a delay

How It Works:

- 1 Enable chat.editing.autoAccept setting
- 2 Set delay time (e.g., 5 seconds)
- 3 Countdown starts after AI completes edits
- 4 Hover over editor controls to cancel countdown
- 5 Changes auto-accept when timer expires



IMPORTANT: Always review changes before committing to source control!

Edit Sensitive Files

Protect critical configuration files



VS Code prompts for approval before editing sensitive files

Commonly Protected Files:



.vscode/*.json

Workspace settings, tasks, launch configs



.env

Environment variables and secrets



package.json

Project dependencies (optional protection)



***.config.js**

Build and framework configurations

```
chat.tools.edits.autoApprove: { "**/*": true, "**/*.env": false }
```

Best Practices

Smart strategies for reviewing AI edits



Review Before Committing

Always check changes before pushing to source control



Check One File at a Time

Review individual files for complex multi-file changes



Understand the Diff

Read inline diffs to understand what changed and why



Use Auto-Accept Wisely

Only for trusted changes or when prototyping



Protect Critical Files

Configure approval for configs, secrets, and settings



Integrate with Git

Stage changes to accept, discard to reject



PLANNING **IN VS CODE CHAT**

Plan before you code - execute with confidence

Why Planning Matters



Structure complex tasks before implementation starts

Review Before Coding

Ensure requirements are clear before changes

Better Guidance

Agent follows systematic approach

Track Progress

Monitor complex tasks with todo lists

Plan first, code later = higher quality results

Plan Agent

Research and create detailed plans

0
1



Built-in agent that creates plans without making code changes

What Plan Agent Does:

- ✓ Research task using read-only tools
- ✓ Break down into actionable steps
- ✓ Identify requirements and constraints
- ✓ Present draft plan for review

✓ No code changes until you approve the plan

How to Use Plan Agent

Four simple steps to create implementation plans

0
2

1

Open Chat View

Press Ctrl+Alt+I and select 'Plan' agent

2

Enter Task

Describe high-level feature, bug, or refactoring

3

Review Plan

Preview draft, provide feedback, iterate

4

Start Implementation

Approve and hand off to default agent

Example Planning Prompts

Real-world examples to get started

Feature

Implement user authentication with OAuth2 and JWT

Testing

Add unit tests for all API endpoints

Bug Fix

Form doesn't reset fields after submission

Refactoring

Migrate legacy components to use hooks

💡 Tip: Stay in plan mode to iterate before coding

What You Get

Structured plan with clear breakdown

0
3

Summary

High-level overview of the implementation

Step Breakdown

Detailed actionable tasks in order

Open Questions

Ambiguous requirements flagged for clarity

Next Actions:

- Start implementation in same chat
- Start background agent session
- Save plan for review or discussion

Todo Lists

Track progress on complex tasks

0
4



Agent creates and updates todos automatically during work

How Todo Lists Work:

- ✓ Agent breaks down task into steps
- ✓ Updates automatically as work completes
- ✓ Helps agent stay focused on goals
- ✓ You can modify using natural language

Example Commands:

```
"Revise step 1 to do x" | "Add another task"
```

Best Practices

Get the most from planning



Research First

Let plan agent understand requirements fully



Iterate the Plan

Refine multiple times before coding starts



Address Open Questions

Clarify ambiguities in the plan draft



Guide Implementation

Provide direction like 'Start with UI'



Monitor Todo Progress

Check completion status during long tasks



USE TOOLS IN CHAT

Extend chat with specialized functionality

What Are Chat Tools?



Tools extend chat with specialized capabilities for tasks

- ✓ Search code and files
- ✓ Run terminal commands
- ✓ Fetch web content
- ✓ Invoke external APIs
- ✓ Edit and create files

Agent chooses tools automatically

Or you can explicitly reference tools with # symbol

Three Types of Tools

Built-in, MCP, and Extension tools

0
1



Built-in Tools

Automatically available, no installation needed -https://code.visualstudio.com/docs/copilot/reference/copilot-vscode-features#_chat-tools

codebase, changes, fetch, githubRepo



MCP Tools

Model Context Protocol servers (local/remote)

Requires installation and configuration



Extension Tools

Contributed by VS Code extensions

Available when extension is installed

MCP - Quick Overview

Model Context Protocol in brief

0
2



Open standard for AI models to use external tools and services

- ✓ Unified interface for tools and data sources
- ✓ Servers can run locally or remotely
- ✓ Requires installation and configuration
- ✓ See dedicated MCP presentation for details



Organizations may restrict which MCP servers can be used

Enable Tools for Chat

Control which tools are available

0
3

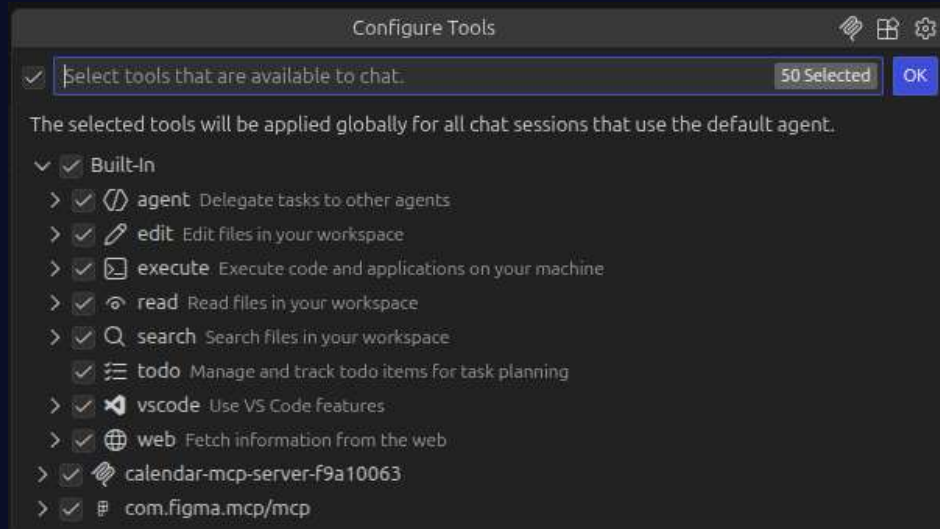
Using the Tools Picker:

- 1 Open Chat view, select Agent mode
- 2 Click Configure Tools button in input field
- 3 Select/deselect tools for this request



Tip:

Select only relevant tools to improve results



Use Tools in Prompts

0
4

Automatic or explicit tool usage

Automatic Selection:

Agent chooses tools based on your prompt

Explicit Reference (with #):

Type # to see available tools

```
"Summarize #fetch https://code.visualstudio.com/updates"
```

```
"How does routing work? #githubRepo vercel/next.js"
```

```
"Fix the issues in #problems"
```

```
"Explain authentication flow #codebase"
```

Tool Approval

Security measure for sensitive operations

0
5



Tools can modify files, run commands, or access services

Approval Scopes:

- Single use (once)
- Current session (until VS Code closes)
- Current workspace (this project)
- All future invocations (always)

⚠️ Always review tool parameters before approving!

Terminal Commands

Agent executes terminal commands

0
6



Commands run in integrated terminal within VS Code



View command output inline in chat



See full output in integrated terminal



Agent terminals marked with chat icon



Auto-approve safe commands (configurable)

```
IONSOLE  TERMINAL  PORTS
> ~/workspace/AITools-Course/ex1$ python3 -m http.server 8080
port 8080 (http://0.0.0.0:8080/) ...
126 22:49:16] "GET /?id=95ca5f41-c804-4d5d-b274-c8816c522b28&vscodeBrows
TP/1.1" 200 -
126 22:49:16] "GET /styles.css HTTP/1.1" 200 -
126 22:49:16] "GET /script.js HTTP/1.1" 200 -
```

Auto-approve example:

```
{ "mkdir": true, "del": false }
```

Tool Sets

Group related tools together



Reference collections of tools as a single entity

Example Tool Set:

```
{
  "reader": {
    "tools": ["changes", "codebase", "problems"],
    "description": "Tools for reading context",
    "icon": "book"
  }
}
```

Use in prompts:

```
"Analyze security issues #reader"
```

0
7

Best Practices

Smart tool usage strategies



Select Relevant Tools Only

Too many tools can reduce effectiveness



Review Approvals Carefully

Check parameters before allowing execution



Use # for Explicit Control

Reference specific tools when needed



Organize with Tool Sets

Group related tools for easier reuse



Configure Auto-Approve Wisely

Only for trusted, safe commands

CUSTOMIZE COPILOT CHAT

Match AI to your workflow with persistent configurations

Why Customize Chat?



Set up persistent configurations that automatically apply your preferred context, tools, and guidelines to every conversation



Save Time

No need to manually provide the same information in each chat request



Ensure Consistency

AI responses automatically align with your coding practices



Flexible Options

Six customization methods that work independently or combined

Six Customization Options

Work independently or combine for comprehensive customization

01

Custom Instructions

Define guidelines in Markdown

02

Agent Skills

Teach specialized capabilities

03

Prompt Files

Reusable task prompts

04

Custom Agents

Specialist assistants

05

Language Models

Choose AI models

06

MCP & Tools

Connect external services

Custom Instructions

Define how tasks should be performed

0
1



Define common guidelines or rules in Markdown files for code generation, reviews, and commits

Use Cases:

- 1 Specify coding practices, technologies, and project requirements
- 2 Provide guidelines for commit messages and PR descriptions
- 3 Set rules for code reviews (security, performance, standards)

Key Features:

Automatic application

Manual inclusion

Agent Skills & Prompt Files

02

Agent Skills (Preview)

Teach specialized capabilities through folders with instructions, scripts, and resources

- ✓ Open standard (agentskills.io) - works across GitHub Copilot tools
- ✓ Portable capabilities across VS Code, CLI, and coding agent
- ✓ VS Code Insiders only (currently)

03

Prompt Files

Reusable prompts for common and repeatable development tasks

- ✓ Create reusable prompts for scaffolding components, APIs, or tests
- ✓ Define code review prompts for quality, security, performance checks
- ✓ Step-by-step guides for complex processes or project patterns

Custom Agents & Language Models

04

Custom Agents

Specialist assistants for specific roles or tasks

- 🗓 Planning agent with read-only access to generate implementation plans
- 🔍 Research agent to explore new technologies and gather information
- 💻 Front-end agent for UI/UX development tasks only

05

Language Models

Choose from different AI models optimized for specific tasks

- ⚡ Fast models for quick code suggestions and simple refactoring
- 🧠 Capable models for complex architectural decisions and reviews
- 🔑 Bring your own API key for experimental or locally hosted models

MCP & Tools

0
6



Connect external services and tools through Model Context Protocol (MCP)



Connect database tools to query and analyze data



Integrate with external APIs for real-time information

Quick Reference:

Project-wide standards



[Custom Instructions](#)

Reusable dev tasks



[Prompt Files](#)

Specialized workflows



[Custom Agents](#)

Complex reasoning



[Language Models](#)

External services



[MCP & Tools](#)

Getting Started Journey

Implement incrementally from simple to complex

1

Try Different Models

Experiment with language models - no setup required

Start here

2

Set Up Guidelines

Create `.github/copilot-instructions.md` for consistency

Quick win

3

Add Task Automation

Build prompt files for repetitive workflows

Time saver

4

Extend Capabilities

Add MCP servers for external service connections

Advanced

5

Create Workflows

Build custom agents for specialized roles

Expert

CUSTOM INSTRUCTIONS

Guide AI behavior with persistent guidelines in VS Code

What are Custom Instructions?



Define common guidelines and rules that automatically influence how AI generates code and handles development tasks

Consistency

Ensure AI responses align with your coding practices

Efficiency

No need to repeat context in every chat prompt

Flexibility

Apply to all requests or specific file types only

⚠ Note: Custom instructions are NOT applied to inline suggestions as you type

Types of Instructions Files

VS Code supports multiple Markdown-based instructions files

01

.github/copilot-instructions.md

Workspace-wide

Single file that applies automatically to all chat requests

02

***.instructions.md**

Conditional

Multiple files with glob patterns for specific file types or locations

03

AGENTS.md

Multi-agent

For workspaces with multiple AI agents (experimental)

04

Settings-based

Specialized

Configure via VS Code settings for specific scenarios

.github/copilot-instructions.md

Single workspace-wide instructions file

0
1

Key Characteristics

- ✓ Automatically applies to ALL chat requests in the workspace
- ✓ Stored at workspace root in .github/ folder
- ✓ Shared with team via version control

Setup Steps:

- 1 Enable github.copilot.chat.codeGeneration.useInstructionFiles setting
- 2 Create .github/copilot-instructions.md at workspace root
- 3 Write instructions in Markdown format using natural language

💡 Also supported by GitHub Copilot in Visual Studio and GitHub.com

*.instructions.md Files

Conditional instructions for specific file types or tasks

Workspace Instructions

Stored in `.github/instructions/`

Only available in current workspace

User Instructions

Stored in user profile

Available across all workspaces

File Format:

Header (YAML Frontmatter):

- description: Short description of the instructions
- name: Display name in UI
- applyTo: Glob pattern defining which files to apply to

Body:

- Custom instructions in Markdown format
- Reference agent tools using `#tool:<tool-name>`

General Coding Guidelines

Markdown



```
---  
applyTo: ""  
---  
  
# Project general coding standards  
  
## Naming Conventions  
- Use PascalCase for component names, interfaces, and type aliases  
- Use camelCase for variables, functions, and methods  
- Prefix private class members with underscore (_)  
- Use ALL_CAPS for constants  
  
## Error Handling  
- Use try/catch blocks for async operations  
- Implement proper error boundaries in React components  
- Always log errors with contextual information
```

Example: Language-specific coding guidelines

Markdown



```
---
applyTo: "**/*.ts,**/*.tsx"
---
# Project coding standards for TypeScript and React

Apply the [general coding guidelines](./general-coding.instructions.md) to all code.

## TypeScript Guidelines
- Use TypeScript for all new code
- Follow functional programming principles where possible
- Use interfaces for data structures and type definitions
- Prefer immutable data (const, readonly)
- Use optional chaining (?.) and nullish coalescing (??) operators

## React Guidelines
- Use functional components with hooks
- Follow the React hooks rules (no conditional hooks)
- Use React.FC type for components with children
- Keep components small and focused
- Use CSS modules for component styling
```

Example: Documentation writing guidelines

Markdown



```
---
applyTo: "docs/**/*.md"
---
# Project documentation writing guidelines

## General Guidelines
- Write clear and concise documentation.
- Use consistent terminology and style.
- Include code examples where applicable.

## Grammar
* Use present tense verbs (is, open) instead of past tense (was, opened).
* Write factual statements and direct commands. Avoid hypotheticals like "could" or "would".
* Use active voice where the subject performs the action.
* Write in second person (you) to speak directly to readers.

## Markdown Guidelines
- Use headings to organize content.
- Use bullet points for lists.
- Include links to related resources.
- Use code blocks for code snippets.
```

Instructions File Example

applyTo: "**/*.ts,**/*.tsx"

name: "TypeScript Guidelines"

description: "TypeScript and React coding standards"

TypeScript Guidelines

Naming Conventions

- Use PascalCase for types and interfaces
- Use camelCase for variables and functions

Error Handling

- Use try/catch for async operations
- Always log errors with context

Creating Instructions Files

Via Chat View

1. Click Configure Chat (gear icon)
2. Select Chat Instructions > New instruction file
3. Choose location (Workspace or User profile)
4. Enter file name and author instructions

Via Command Palette

1. Open Command Palette (Ctrl+Shift+P)
2. Run 'Chat: New Instructions File'
3. Choose location and name
4. Edit the file with your instructions

💡 Tip: Store project-specific instructions in workspace to share with team

Additional Features



Auto-Generate Instructions

VS Code can analyze your workspace and generate matching instructions automatically

[Configure Chat > Generate Chat Instructions](#)



Sync Across Devices

Use Settings Sync to share user instructions files across multiple devices

[Settings Sync > Configure > Prompts and Instructions](#)



AGENTS.md Support

Define instructions for multiple AI agents with nested folder support

[Enable chat.useAgentsMdFile setting](#)

Best Practices

DO ✓

- ✓ Keep instructions short and focused
- ✓ Use multiple files for different topics
- ✓ Store project-specific in workspace
- ✓ Reference other instruction files
- ✓ Use applyTo patterns wisely
- ✓ Test instructions with examples

DON'T ✗

- ✗ Write overly long instructions
- ✗ Duplicate instructions across files
- ✗ Forget to enable required settings
- ✗ Use vague or ambiguous language
- ✗ Apply to wrong file patterns
- ✗ Ignore instruction conflicts



PROMPT FILES IN VS CODE

Reusable prompts for standardized workflows

What Are Prompt Files?



Markdown files defining reusable prompts for development tasks

- Generate code components
- Perform code reviews
- Scaffold project structures
- Run security audits
- Create documentation

Key Difference:

Unlike custom instructions (apply to all), prompt files trigger on-demand

Two Types of Scope

Choose where prompts are available

0
1



Workspace Prompt Files

📁 Stored in .github/prompts folder

✓ Only available within workspace



User Prompt Files

📁 Stored in VS Code profile folder

✓ Available across all workspaces

Prompt File Structure

Extension: .prompt.md

0
2

YAML Header (optional):

- description, name, argument-hint
- agent (ask, edit, agent, custom)
- model (GPT-4o, Claude Sonnet 4, etc.)
- tools (list of available tools)

Body:

Contains prompt text sent to LLM with instructions and guidelines

Can include:

- Markdown links to workspace files
- Variables: \${workspaceFolder}, \${selection}
- Tool references: #tool:githubRepo
- Input variables: \${input:variableName}

Example: React Form

Complete prompt file structure

```
agent: 'agent'
model: GPT-4o
tools: ['githubRepo', 'search/codebase']
description: 'Generate a new React form component'
---
```

Your goal is to generate a new React form component based on the templates in #tool:githubRepo contoso/react-templates.

Ask for the form name and fields if not provided.

Requirements for the form:

- * Use form design system components: [design-system/Form.md](../docs/design-system/Form.md)
- * Use `react-hook-form` for form state management:
- * Always define TypeScript types for your form data
- * Prefer `uncontrolled` components using register
- * Use `defaultValues` to prevent unnecessary rerenders
- * Use `yup` for validation:
- * Create reusable validation schemas in separate files
- * Use TypeScript types to ensure type safety
- * Customize UX-friendly validation rules

Example: Example: perform a security review of a REST API

Complete prompt file structure

```
---
agent: 'ask'
model: Claude Sonnet 4
description: 'Perform a REST API security review'
---
Perform a REST API security review and provide a TODO list of security issues to address.
* Ensure all endpoints are protected by authentication and authorization
* Validate all user inputs and sanitize data
* Implement rate limiting and throttling
* Implement logging and monitoring for security events
Return the TODO list in a Markdown format, grouped by priority and issue type.
```

Create a Prompt File

Four simple steps

0
3

1

Open Menu

Configure Chat > Prompt Files > New

2

Choose Location

Workspace or User profile

3

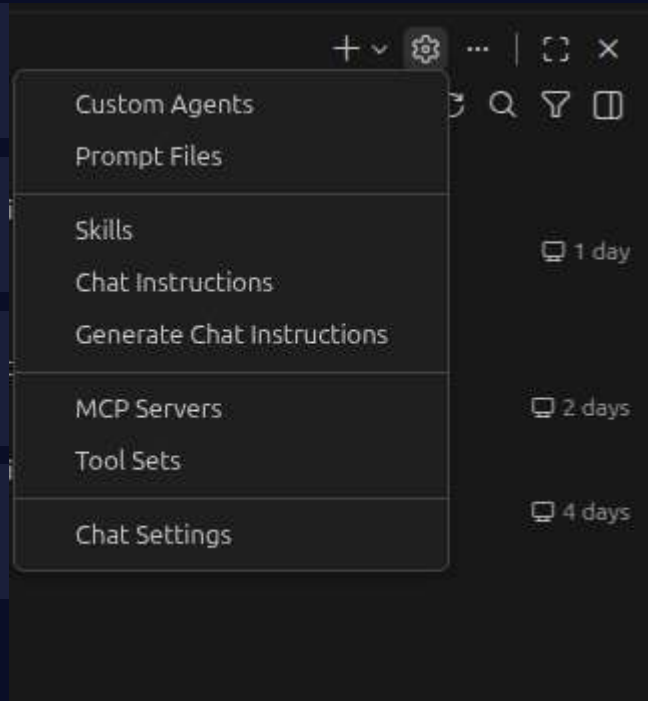
Enter File Name

Name appears when typing / in chat

4

Author Prompt

Fill YAML frontmatter and body



Use Prompt Files

Three ways to run prompts

0
4



Type / in Chat

Type /prompt-name with optional parameters

/create-react-form formName=MyForm



Command Palette

Chat: Run Prompt > Select from list

Quick access to all prompts



Editor Play Button

Open prompt file > Click play in title bar

Test and iterate quickly

Tool List Priority

Order of tool selection

0
5



When both custom agent and prompt file specify tools

Priority Order:

1

Tools specified in prompt file (highest)

2

Tools from referenced custom agent

3

Default tools for selected agent (lowest)

Sync Across Devices

Keep prompts in sync with Settings Sync



User prompt files can sync across multiple devices

Setup Steps:

- 1 Enable Settings Sync in VS Code
- 2 Run Settings Sync: Configure command
- 3 Select 'Prompts and Instructions' to sync

💡 Workspace prompts (.github/prompts) are not synced

0
6

Best Practices

Tips for effective prompts



Clear Instructions

Describe goals and expected output format



Provide Examples

Include input/output samples



Reference Custom Instructions

Use Markdown links vs duplicating



Use Variables

`${selection}` and input variables



Test with Play Button

Iterate and refine based on results



CUSTOM AGENTS **IN VS CODE**

AI personas tailored for specialized development roles

What Are Custom Agents?



AI personas with specific behaviors, tools, and instructions

An Agent Consists Of:

- ✓ Custom instructions defining behavior
- ✓ Specific set of available tools
- ✓ Tailored for specialized development roles
- ✓ Stored as .agent.md Markdown files

Example Personas:

Planner:	Generate implementation plans
Security Reviewer:	Audit code for vulnerabilities
Architect:	Design system architecture
Code Reviewer:	Review PRs and suggest improvements

Why Use Custom Agents?

Right tools for the right job

0
1

Task-Appropriate Tools

Planners use read-only, implementers use full editing

Specialized Instructions

Define exactly how AI should operate for each role

Consistent Responses

Same behavior every time you switch to that agent

Reusable Across Projects

Use in background and cloud agents too

Handoffs

Sequential workflows with suggested next steps

0
2



Transition between agents with context and pre-filled prompts

Example Workflows:

Planning → Implementation → Review

Write Failing Tests → Write Passing Tests

Architecture Design → Code Implementation

Security Audit → Fix Issues

How it works:

Handoff buttons appear after response → Click to switch agents with pre-filled prompt

Agent File Structure

Extension: .agent.md

0
3

YAML Header (optional):

- description, name, argument-hint
- tools (list of available tools)
- model (GPT-4o, Claude Sonnet 4, etc.)
- handoffs (label, agent, prompt, send)
- infer, target, mcp-servers

Body:

Contains instructions and guidelines for AI behavior in Markdown format

- Reference files with Markdown links
- Use tool references: #tool:githubRepo
- Prepend to user chat prompts
- Can reuse instruction files

Example: Planner Agent

Complete agent file structure

```
---  
description: Generate implementation plan  
name: Planner  
tools: ['fetch', 'githubRepo', 'search']  
model: Claude Sonnet 4  
handoffs:  
  - label: Implement Plan  
    agent: agent  
    prompt: Implement the plan outlined above.  
---
```

```
# Planning instructions  
Generate an implementation plan.  
Don't make code edits, just create a plan with:  
* Overview and requirements  
* Implementation steps  
* Testing strategy
```

Create a Custom Agent

Four simple steps

0
4

1

Open Menu

Configure Custom Agents > Create new

2

Choose Location

Workspace (.github/agents) or User profile

3

Enter File Name

Name appears in agents dropdown

4

Define Agent

Fill YAML header and body instructions

Tool List Priority

How tools are determined

0
5



Both custom agents and prompt files can specify tools

Priority Order:

1

Tools in prompt file (highest priority)

2

Tools from custom agent in prompt file

3

Default tools for selected agent (lowest)

Share Across Teams

Organization-level custom agents (Experimental)

0
6



User Level

Personal agents across all workspaces

 *User profile folder*



Workspace Level

Team agents in .github/agents folder

 *Shared via repository*



Organization Level

Org-wide agents for all members

 *GitHub organization (experimental)*

Customize Dropdown

Show or hide agents in dropdown list

0
7



Control which custom agents appear in agents dropdown

Steps:

1

Select 'Configure Custom Agents' from dropdown

2

Hover over agent in list

3

Click eye icon to show/hide in dropdown



Use Case:

Hide extension-contributed agents without uninstalling the extension

Best Practices

Create effective custom agents



Clear Role Definition

Define specific purpose and behavior



Appropriate Tools

Read-only for planners, full edit for implementers



Use Handoffs

Create guided workflows between agents



Test and Iterate

Refine instructions based on results



Share with Team

Use workspace agents for consistency



HOW TO CREATE HANDOFFS

Step-by-step tutorial with examples

Before You Start

What you need

Requirements:

- 1 VS Code installed with GitHub Copilot extension
- 2 At least two custom agents created (.agent.md files)
- 3 Basic understanding of YAML format
- 4 Agent files in .github/agents folder (workspace) or user profile

Example Scenario We'll Build:

Planning Agent → Implementation Agent

User clicks button after planning to start implementation

Context flows: plan details → implementation prompt

Step 1: Locate Agent File

0
1

Find the agent file you want to add handoffs to

Method 1: Via Chat Interface

- Open Chat View in VS Code
- Click agents dropdown (top of chat)
- Select 'Configure Custom Agents'
- Click on your planning agent from the list

Method 2: Direct File Access

- Navigate to `.github/agents/planning.agent.md`
- Or your user profile agents folder

File Location Example:

```
.github/agents/planning.agent.md
```

Step 2: View Current File

See what the file looks like before adding handoffs

0
2

Original Planning Agent (BEFORE):

```
---  
description: Generate an implementation plan  
name: Planning Agent  
tools: ['search', 'fetch', 'codebase']  
model: Claude Sonnet 4  
---  
  
# Planning instructions  
You are in planning mode. Generate a detailed  
implementation plan including:  
* Overview and requirements  
* Step-by-step implementation guide  
* Testing strategy  
  
Do NOT make any code changes.
```

This agent works fine but has NO handoffs yet

Step 3: Add Handoffs

Section

Add the handoffs field to YAML frontmatter

0
3

What to do:

- 1 Place cursor after the last field in YAML frontmatter (before ---)
- 2 Add a new line
- 3 Type: handoffs:
- 4 Press Enter to start defining handoff items

File with handoffs: added (NEW):

```
---
description: Generate an implementation plan
name: Planning Agent
tools: ['search', 'fetch', 'codebase']
model: Claude Sonnet 4
handoffs:
  - label: Start Implementation
    agent: implementation
    prompt: Now implement the plan above.
    send: false
---
```

Step 4: Define Handoff Properties

0
4

Fill in the four required fields

label: Text shown on the button

Example: 'Start Implementation'

Tip: Use action verbs, keep it short

agent: Target agent identifier

Example: 'implementation'

Tip: Must match agent name exactly

prompt: Message sent to next agent

Example: 'Now implement the plan outlined above.'

Tip: Reference previous work

send: Auto-submit the prompt?

Example: false (default: user reviews first)

Tip: Use false for safety

Step 5: Complete Example

Full agent file with handoff

0
5

```
---
description: Generate an implementation plan
name: Planning Agent
tools: ['search', 'fetch', 'codebase']
model: Claude Sonnet 4
handoffs:
  - label: Start Implementation
    agent: implementation
    prompt: |
      Now implement the plan outlined above.
      Follow the architecture decisions and
      use the recommended libraries.
    send: false
---

# Planning instructions
You are in planning mode. Generate a detailed
implementation plan. Do NOT make code changes.
```

Step 6: Save and Test

0
6

Save file and test the handoff



Save the File

Press Ctrl+S (Windows/Linux) or Cmd+S (Mac)

Testing Steps:

- 1 Open Chat View in VS Code
- 2 Switch to your Planning Agent
- 3 Ask it to create a plan (e.g., 'Plan a login feature')
- 4 Wait for response to complete
- 5 Look for the 'Start Implementation' button below response
- 6 Click the button - you should switch to Implementation agent
- 7 Verify the prompt is pre-filled with your message

Adding Multiple Handoffs

Offer multiple next steps

0
7



You can add multiple handoff options - just add more items to the list

```
handoffs:
```

- label: Start Implementation
agent: implementation
prompt: Implement the plan above.
send: false
- label: Review Architecture
agent: architect
prompt: Review this plan for issues.
send: false
- label: Create Tests First
agent: test-writer
prompt: Write tests based on this plan.
send: false

Result: Three buttons appear after planning response

Common Mistakes to Avoid

Watch out for these errors

✗ Wrong indentation in YAML

✓ Fix: Use exactly 2 spaces for each level, no tabs

- label: (correct) vs - label: (wrong)

✗ Agent name doesn't match

✓ Fix: Agent identifier must match exactly (case-sensitive)

'implementation' not 'Implementation Agent'

✗ Missing quotes in strings

✓ Fix: Wrap text values in single or double quotes

label: 'Start' not label: Start

✗ Handoffs outside YAML section

✓ Fix: Must be inside --- markers at top of file

Place before closing --- line

Troubleshooting

What to do when things don't work

Problem: Handoff button doesn't appear

Solutions:

- Check YAML syntax is correct (use YAML validator)
- Ensure file is saved (Ctrl+S / Cmd+S)
- Restart VS Code if needed
- Verify agent completed its response fully

Problem: Button appears but nothing happens

Solutions:

- Check target agent name matches exactly
- Ensure target agent exists and is accessible
- Look for errors in VS Code Developer Tools
- Try with `send: false` first for debugging

Pro Tip: Test with `send: false` first, then switch to true when working

Quick Reference

Handoff template to copy

```
handoffs:  
  - label: 'Your Button Text Here'  
    agent: 'target-agent-name'  
    prompt: 'Your message to next agent'  
    send: false
```

Remember:

- ✓ label: Text shown on button (action-oriented)
- ✓ agent: Must match target agent name exactly
- ✓ prompt: Message with context for next agent
- ✓ send: false (review first) or true (auto-submit)

Practice Exercise

Try creating this handoff

Exercise: Implementation → Review Workflow

After implementation completes, add a handoff button that:

1. Shows button labeled 'Review My Code'
2. Switches to a 'code-reviewer' agent
3. Sends message: 'Please review the code I just implemented'
4. Requires user to review prompt before sending

Solution on next slide (try first!)

Hints:

- Remember to use quotes around text values
- Agent name is 'code-reviewer' (lowercase, with hyphen)
- send: false means user reviews before sending
- Place in implementation.agent.md file

Exercise Solution

How did you do?

```
---
description: Implement code based on plan
name: Implementation Agent
tools: ['edit', 'codebase', 'terminal']
model: Claude Sonnet 4
handoffs:
  - label: 'Review My Code'
    agent: 'code-reviewer'
    prompt: 'Please review the code I just implemented'
    send: false
---

# Implementation instructions
Implement the planned features with full editing tools.
```

This creates a handoff from implementation to code review!



AGENT **SKILLS**

Specialized capabilities with instructions and resources

What Are Agent Skills?



Folders with instructions, scripts, and resources for specialized tasks

A Skill Contains:

- ✓ SKILL.md file with instructions
- ✓ Scripts and automation tools
- ✓ Example files and templates
- ✓ Documentation and resources
- ✓ All stored in a single directory

Key Benefits:

- Specialize Copilot for domain-specific tasks
- Reduce repetition across conversations
- Compose multiple skills for complex workflows
- Load only relevant content when needed

Skills vs Custom Instructions

Different tools for different purposes

0
1

Purpose

Skills: Specialized capabilities and workflows

Instructions: Coding standards and guidelines

Portability

Skills: VS Code, CLI, Copilot coding agent

Instructions: VS Code and GitHub.com only

Content

Skills: Instructions, scripts, examples, resources

Instructions: Instructions only

Scope

Skills: Task-specific, loaded on-demand

Instructions: Always applied (or via glob patterns)

Standard

Skills: Open standard (agentskills.io)

Instructions: VS Code-specific

When to Use Skills

Choose the right tool for your needs

0
2

Use Agent Skills When:

- ✓ Creating reusable capabilities across AI tools
- ✓ Including scripts, examples, or resources
- ✓ Sharing capabilities with the AI community
- ✓ Defining specialized workflows (testing, debugging, deployment)

Use Custom Instructions When:

- ✓ Defining project-specific coding standards
- ✓ Setting language or framework conventions
- ✓ Specifying code review or commit guidelines
- ✓ Applying rules based on file types with glob patterns

Skill Directory Structure

Where skills are stored

0
3



Project Skills

`.github/skills/`

Stored in repository, shared with team



Personal Skills

`~/.copilot/skills/`

Stored in user profile, available across projects

Example Skill Directory:

```
.github/skills/webapp-testing/  
├─ SKILL.md           # Main instructions  
├─ test-template.js   # Template file  
└─ examples/         # Example scenarios  
    ├─ login-test.js  
    └─ checkout-test.js
```

SKILL.md File Format

0
4

Two parts: YAML header and markdown body

YAML Header (Required):

name

Unique identifier (lowercase, hyphens, max 64 chars)

description

What it does AND when to use it (max 1024 chars)

Markdown Body:

Contains instructions that describe:

- What the skill helps accomplish
- When to use the skill
- Step-by-step procedures to follow
- Examples of expected input and output
- References to included scripts or resources

Example: Testing Skill

Complete SKILL.md file

```
name: webapp-testing
description: Guide for testing web applications using Playwright. Use this when asked to create or
run browser-based tests.
---
# Web Application Testing with Playwright
This skill helps you create and run browser-based tests for web applications using Playwright.
## When to use this skill
Use this skill when you need to:
- Create new Playwright tests for web applications
- Debug failing browser tests
- Set up test infrastructure for a new project
## Creating tests
1. Review the [test template](./test-template.js) for the standard test structure
2. Identify the user flow to test
3. Create a new test file in the `tests/` directory
4. Use Playwright's locators to find elements (prefer role-based selectors)
5. Add assertions to verify expected behavior
## Running tests
To run tests locally:
```bash
npx playwright test
```
To debug tests:
```bash
npx playwright test --debug
```
## Best practices
- Use data-testid attributes for dynamic content
- Keep tests independent and atomic
- Use Page Object Model for complex pages
- Take screenshots on failure
```

How Copilot Uses Skills

0
5

Progressive disclosure: three-level loading

1

Skill Discovery

Reads name and description from all skills

When: Always available, lightweight metadata

2

Instructions Loading

Loads SKILL.md body into context

When: When request matches skill description

3

Resource Access

Accesses scripts, examples, documentation

When: Only when Copilot references them

Skills activate automatically based on your prompt - no manual selection needed

Create a Skill

Four simple steps

0
6

1

Create Directory

Make `.github/skills` directory in workspace

2

Create Subdirectory

One directory per skill (e.g., `webapp-testing`)

3

Create SKILL.md

Add YAML header (name, description) and body

4

Add Resources (Optional)

Include scripts, templates, examples

Use Shared Skills

Leverage community-created skills

0
7

Community Repositories:



[github/awesome-copilot-](https://github.com/github/awesome-copilot) <https://github.com/github/awesome-copilot>

Skills, agents, instructions, and prompts



[anthropics/skills](https://www.aitmpl.com/hooks) - <https://www.aitmpl.com/hooks>

Reference skills collection

How to Use Shared Skills:

1

Browse available skills in repository

2

Copy skill directory to your .github/skills/ folder

3

Review and customize SKILL.md for your needs

4

Optionally modify or add resources

Always review shared skills before use for security and quality

Open Standard

Portable across multiple AI agents

0
8



Agent Skills is an open standard at agentskills.io

Works With Multiple Agents:

GitHub Copilot in VS Code

Available in chat and agent mode

GitHub Copilot CLI

Accessible when working in terminal

GitHub Copilot coding agent

Used during automated coding tasks

Key Benefit: Write once, use everywhere

Best Practices

Create effective skills



Clear Descriptions

Explain what AND when - helps Copilot decide when to load



Step-by-Step Instructions

Provide clear procedures with examples



Include Resources

Add templates, scripts, and examples



Test Your Skills

Verify they work with different prompts



Share with Community

Contribute to awesome-copilot repository

AGENTS vs SKILLS

Two Building Blocks for AI-Assisted Work

Two Different Things

A Skill is knowledge. An Agent is a doer. They are not the same kind of thing.



SKILL

Reusable knowledge

- Packaged know-how, conventions, and procedures
- Passive — it describes HOW to do something
- Loaded into a model when relevant
- Static and consistent every time



AGENT

Autonomous doer

- Pursues a goal end-to-end on its own
- Active — it actually DOES the work
- Plans, runs tools, checks, and self-corrects
- Dynamic — reacts to its environment

Recipe vs Chef

The clearest way to picture the difference



A Skill is the recipe

A well-written set of instructions. It sits there and waits — it does nothing by itself, but it tells you exactly how to make the dish well.



An Agent is the chef

Walks into the kitchen, reads recipes, tastes as they go, fixes the seasoning, decides when it's done. Acts on its own until the goal is met.

The chef *uses* the recipes. The two work together.

In Code Terms

How the distinction maps to things you already build



Skill $\approx$ conventions file

Like a Helper class or a coding-standards doc: static knowledge you wrote once and pull in whenever you need it.

```
// SKILL.md
xUnit + Moq + FluentAssertions
“generate tests like this”
```



Agent $\approx$ BackgroundService

Like a worker that runs a loop: takes a goal, acts, checks the result, corrects, repeats — until the job is done.

```
while (!goalMet) {
    act(); check(); fix();
}
```

At a Glance

Pin this to the wall

| | SKILL | AGENT |
|----------------|---------------------|---------------------------|
| Nature | Passive knowledge | Active doer |
| Does anything? | No — it only knows | Yes — it executes |
| How it runs | Loaded into a model | Runs in its own loop |
| State | Static, fixed | Dynamic, reacts |
| Analogy | The recipe | The chef |
| Example | "How we write docx" | "Fix the bugs, open a PR" |

Which Do I Use?

A simple rule of thumb



Reach for a SKILL

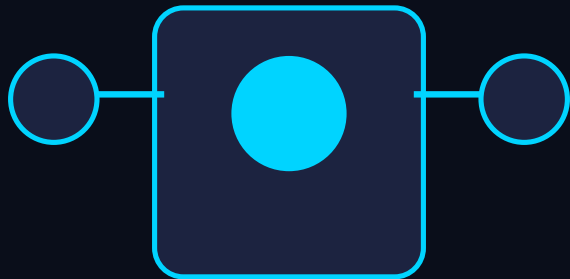
when you want to capture reusable know-how — conventions, formats, procedures — so it's applied consistently without re-explaining it each time.



Reach for an AGENT

when something must run a task autonomously end-to-end — planning, using tools, checking results, and correcting itself until the goal is reached.

And in practice they combine: the agent does the work, the skills tell it how to do each part right.



MODEL CONTEXT PROTOCOL

The universal standard for connecting AI to everything

What is MCP?

MCP

An open-source standard for connecting AI applications to external systems

Key Capabilities:

- ✓ Connect to data sources — local files, databases, cloud storage
- ✓ Integrate with tools — search engines, calculators, APIs
- ✓ Access specialized workflows and AI-powered prompts
- ✓ Enable agents to access information and perform actions
- ✓ Works with Claude, ChatGPT, and other AI applications

Think of MCP like a USB-C port for AI — one universal standard for everything

Why MCP Matters

Benefits across the ecosystem

0
2



Reduces development time and complexity when building AI applications or integrating external tools



Provides access to an ecosystem of data sources, tools and apps that enhance capabilities and user experience



Results in more capable AI that can access your data and take actions on your behalf automatically

Without MCP:

Every AI integration required custom, one-off connections — complex and hard to maintain

How MCP Works

Architecture overview

03



Servers provide:

- Tools — actions AI can invoke (e.g. search, write file)
- Resources — data AI can read (e.g. docs, DBs)
- Prompts — reusable workflow templates

Clients can use:

- Discover available tools and resources
- Execute tool calls on behalf of the AI
- Manage context and conversation flow

Real-World Examples

What MCP can enable today

CALENDAR + NOTES

Personal AI Assistant

Agents access Google Calendar and Notion, becoming a fully personalized AI assistant that knows your schedule and notes

FIGMA → CODE

Design to Code

Claude Code generates an entire web app directly from a Figma design file — bridging design and development workflows

DATABASES

Enterprise Data Access

Enterprise chatbots connect to multiple databases across an organization, enabling data analysis via natural language

BLENDER + PRINTER

3D Design & Print

AI models create 3D designs in Blender and send them directly to a 3D printer — physical world automation

Start Building with MCP

0
4

Three paths to get started

1

Build an MCP Server

Create an MCP server to expose your data sources and tools. Use the official SDKs for Python, TypeScript, or other supported languages.

2

Build an MCP Client

Develop an application that connects to MCP servers. Your client can discover tools and resources, then invoke them on behalf of users.

3

Connect & Explore

Use the MCP Inspector to debug and test your integrations. Browse the MCP Registry to find existing servers and connect to them.

docs: modelcontextprotocol.io/docs/getting-started/intro

SUBAGENTS

Delegating Work to Specialized Child Agents

What Is a **Subagent**?

A specialized agent that the main agent hands a focused task to



An agent can delegate a side task to a child agent that runs in its own isolated context, with its own tools and even its own model — then returns just a summary.



Own context window

Starts fresh — doesn't see the main conversation. Verbose work stays out of the parent.



Scoped tools & model

A curated tool allowlist and its own model (e.g. cheap Haiku for read-only search).



Returns a summary

The parent gets the conclusion, not every file read or log line.

Where It Fits

Same family as Skills and Agents — but solving a different problem

| | SKILL | AGENT | SUBAGENT |
|------------|-----------------------|------------------------|---------------------------------|
| What it is | Reusable knowledge | Autonomous doer | A delegated child doer |
| Context | Loaded into the model | The main context | Its own isolated window |
| Runs where | In the main thread | In the main thread | In a separate thread |
| Best for | Applying conventions | Driving the whole task | Noisy, self-contained side work |

Rule of thumb: a subagent IS an agent — just one the main agent spawns to keep its own context clean.

Why Delegate to a Subagent?

Worth the overhead when one of these actually helps



Preserve context

Keep verbose searches, test output, and dead-ends out of the main window.



Run in parallel

Independent investigations run at once — each reports back a summary.



Enforce constraints

Lock a subagent to read-only tools, or scope it to specific MCP servers.



Fresh perspective

A clean-context reviewer catches what the main session stopped seeing.

Defining a Subagent

A Markdown file with YAML frontmatter — just like a SKILL.md

```
~/.claude/agents/code-reviewer.md
```

```
---
```

```
name: code-reviewer
```

```
description: Reviews recent  
             changes. Use proactively.
```

```
tools: Read, Grep, Glob, Bash
```

```
model: haiku
```

```
---
```

```
You are a senior code reviewer.  
Focus on quality and security...
```

name

how you @-mention or invoke it

description

the trigger — tells the agent when to delegate

tools

an allowlist; omit to inherit everything

model

route to a cheaper model to control cost

body

becomes the subagent's system prompt

Use Them **Deliberately**

Subagents carry overhead — reach for them on purpose, not by default



Reach for a subagent

- Verbose, self-contained side work
- Independent tasks that can run in parallel
- You want tool/permission restrictions
- You need a fresh, unbiased review



Stay in the main thread

- Sequential work where each step needs the last
- Edits to the same files (they collide)
- Quick, targeted changes
- Tight back-and-forth iteration

Note: a subagent can't spawn another subagent — chain them from the main thread instead.

Long Flow: The Pipeline

An orchestrator delegates down a chain of five workers — it never writes code itself

